

Reviewer Pack — Schur-Weyl Multiplicity Gap in Symmetric vs Antisymmetric De...

Entry 38826d199a13 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 04:24:33 UTC

Schur-Weyl Multiplicity Gap in Symmetric vs Antisymmetric Decompositions

Entry ID: 38826d199a13 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 04:24:33 UTC

1. Conjecture

Field A (mathematical branch): SCHUR_WEYL_DUALITY **Field B** (complexity object): GEOMETRIC_COMPLEXITY_THEORY

Statement:

For all integers $n \geq 2$, the multiplicity of the irreducible representation indexed by the partition $\lambda = (n-1, 1)$ in the decomposition of the symmetric square of the standard representation of $GL(n)$ is strictly greater than its multiplicity in the decomposition of the antisymmetric square of the standard representation.

Rationale (proposer's reasoning):

The symmetric square decomposition (related to the permanent) and antisymmetric square decomposition (related to the determinant) exhibit distinct representation-theoretic structures. By comparing multiplicities of key irreducible components (e.g., $\lambda = (n-1, 1)$), we isolate a quantitative invariant preserved under linear substitutions. A gap in these multiplicities could imply structural differences in their orbit closures, advancing the GCT program's goal to separate det and perm.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4ca4054ae4fdaaf5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import math
import random
from fractions import Fraction

def hook_length_formula(n, partition):
    total = 1
    for i in range(n):
        for j in range(len(partition)):
            if partition[j] == 0:
                continue
            total *= (n - i + j) / (partition[j] * (i + j))
            partition[j] -= 1
    return total

def multiplicity(n, partition):
    sym_multiplicity = hook_length_formula(n, partition[:]) / math.factorial(n)
    antisym_multiplicity = hook_length_formula(n, partition[:]) / math.factorial(n)
    return sym_multiplicity, antisym_multiplicity

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        if n < 2:
            continue
        sym_multiplicity, antisym_multiplicity = multiplicity(n, (n-1, 1))
        results.append((sym_multiplicity, antisym_multiplicity))

    if not results:
        return {
            "metric_name": "Multiplicity Gap",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
```

```

    }

    sym_total = sum(result[0] for result in results)
    antisym_total = sum(result[1] for result in results)
    mean_sym = sym_total / len(results)
    mean_antisym = antisym_total / len(results)

    return {
        "metric_name": "Multiplicity Gap",
        "metric_value": mean_sym - mean_antisym,
        "instances_tested": len(results),
        "conjecture_holds": mean_sym > mean_antisym,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no trials executed")
        sys.exit(0)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Multiplicity gap does not hold\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no support for conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115dcd49.py", line 72, in <module>
    trial_result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115dcd49.py", line 41, in run_trial
    sym_multiplicity, antisym_multiplicity = multiplicity(n, (n-1, 1))
    ~~~~~

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115dcd49.py", line 29, in multiplicity
    sym_multiplicity = hook_length_formula(n, partition[:]) / math.factorial(n)
                        ~~~~~^~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_115dcd49.py", line 24, in hook_length_f
    total *= (n - i + j) / (partition[j] * (i + j))
            ~~~~~^~~~~~
```

```
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

```
parse_fail | next: NONE
```

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107338
2	preregistration	ollama_remote	qwen3:8b	0	0	27823
3	novelty	ollama_remote	qwen3:8b	0	0	24046
4	novelty	ollama_remote	qwen3:8b	0	0	17472
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13527
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10686
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8449
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9866
9	judge	ollama_remote	qwen3:8b	0	0	26890

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 246096 ms total latency. Provider mix: {'ollama remote': 9}

(full prompt+response transcripts available in [research/audit/38826d199a13.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/38826d199a13.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp sandbox/test *.py` (per-cycle)

Replay tarball: `research/replay/38826d199a13.tar.gz` (if generated)